

IV_SEM\Lab_1\BisectionMethod.py

```
1  """
2      By      :- Darshan Shrestha
3      Roll no.:- KCE081BCT010
4  """
5  import matplotlib.pyplot as plt
6  import numpy as np
7
8  TOLERABLE_ERROR = 10**-5
9
10 def func(x):
11     return x**2 - 5 * x + 5
12
13
14 def plotFunc():
15     x_values = np.linspace(0, 5, 200)
16     y_values = [func(x) for x in x_values]
17
18     plt.figure(figsize=(8, 4))
19     plt.plot(x_values, y_values, label="f(x) = x^2 - 5x + 5")
20     plt.axhline(0, color="black", linewidth=0.8)
21     plt.xlabel("x")
22     plt.ylabel("f(x)")
23     plt.title("Plot of f(x)")
24     plt.grid(True)
25     plt.legend()
26     plt.show()
27
28 def print_iteration_table(rows) -> None:
29     columns = ["Iteration", "a", "b", "m", "f(a)", "f(m)", "Error"]
30     widths = {
31         "Iteration": len("Iteration"),
32         "a": len("a"),
33         "b": len("b"),
34         "m": len("m"),
35         "f(a)": len("f(a)"),
36         "f(m)": len("f(m)"),
37         "Error": len("Error"),
38     }
39
40     for row in rows:
41         widths["Iteration"] = max(widths["Iteration"], len(f"{row['Iteration']}"))
42         for key in ("a", "b", "m", "f(a)", "f(m)", "Error"):
43             widths[key] = max(widths[key], len(f"{row[key]:.6f}"))
44
45     header = " | ".join(f"{column:>{widths[column]}}" for column in columns)
46     separator = "-+-".join("-" * widths[column] for column in columns)
47
48     print("\nBy      :- Darshan Shrestha")
49     print("Roll no. :- KCE081BCT010")
50     print("\nIteration Table:\n")
51     print(header)
```

```

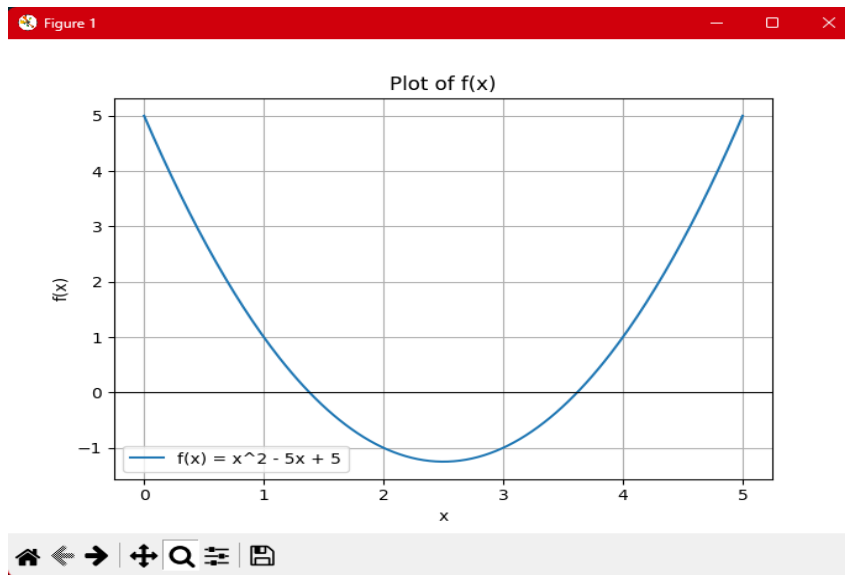
52 print(separator)
53 for row in rows:
54     print(
55         f"{row['Iteration']:>{widths['Iteration']}} | "
56         f"{row['a']:>{widths['a']}.6f} | "
57         f"{row['b']:>{widths['b']}.6f} | "
58         f"{row['m']:>{widths['m']}.6f} | "
59         f"{row['f(a)']:>{widths['f(a)']}.6f} | "
60         f"{row['f(m)']:>{widths['f(m)']}.6f} | "
61         f"{row['Error']:>{widths['Error']}.6f}"
62     )
63
64 def findRoot(a, b) -> list:
65     iterations = 0
66     rows = []
67     while True:
68         m = (a + b) / 2
69         func_a = func(a)
70         func_m = func(m)
71         error = abs(b - a) / 2
72
73         rows.append({
74             "Iteration": iterations + 1,
75             "a": a,
76             "b": b,
77             "m": m,
78             "f(a)": func_a,
79             "f(m)": func_m,
80             "Error": error,
81         })
82
83         if func_m * func_a < 0:
84             b = m
85         else:
86             a = m
87
88         iterations += 1
89         if abs(func_m) < TOLERABLE_ERROR:
90             return [m, iterations, rows]
91
92 def main() -> None:
93     check = True
94     plotFunc();
95     while check:
96         print("f(x) = x**2 - 5 * x + 5\n")
97         initGuess_1, initGuess_2 = map(float, input("Enter initial guesses"
98             "(2 only; separate with spaces).=> ").split())
99
100         if (func(initGuess_1) * func(initGuess_2) >= 0):
101             print("Error; the product of the functional value of inputs must be strictly less than 0.\n")
102         else:
103             check = False
104
105     a = initGuess_1

```

```

106 b = initGuess_2
107
108 root, iterations, rows = findRoot(a,b)
109 print_iteration_table(rows)
110
111 root_line = f"Root: {root:.5f}"
112 iter_line = f"Ran {iterations} iterations"
113 content_width = max(len(root_line), len(iter_line))
114 border = "+" + "-" * (content_width + 2) + "+"
115 print()
116 print(border)
117 print(f"| {root_line.ljust(content_width)} |")
118 print(f"| {iter_line.ljust(content_width)} |")
119 print(border)
120
121
122 if __name__ == "__main__":
123     main()
124

```



```

C:\WINDOWS\System32\cmd  X  +  v
f(x) = x**2 - 5 * x + 5
Enter initial guesses(2 only; separate with spaces).=> 1.38 1.39
By          :- Darshan Shrestha
Roll no.    :- KCE081BCT010

Iteration Table:
Iteration | a | b | m | f(a) | f(m) | Error
-----|-----|-----|-----|-----|-----|-----
1 | 1.380000 | 1.390000 | 1.385000 | 0.004400 | -0.006775 | 0.005000
2 | 1.380000 | 1.385000 | 1.382500 | 0.004400 | -0.001194 | 0.002500
3 | 1.380000 | 1.382500 | 1.381250 | 0.004400 | 0.001602 | 0.001250
4 | 1.381250 | 1.382500 | 1.381875 | 0.001602 | 0.000204 | 0.000625
5 | 1.381875 | 1.382500 | 1.382187 | 0.000204 | -0.000495 | 0.000312
6 | 1.381875 | 1.382187 | 1.382031 | 0.000204 | -0.000146 | 0.000156
7 | 1.381875 | 1.382031 | 1.381953 | 0.000204 | 0.000029 | 0.000078
8 | 1.381953 | 1.382031 | 1.381992 | 0.000029 | -0.000059 | 0.000039
9 | 1.381953 | 1.381992 | 1.381973 | 0.000029 | -0.000015 | 0.000020
10 | 1.381953 | 1.381973 | 1.381963 | 0.000029 | 0.000007 | 0.000010

+-----+
| Root: 1.38196 |
| Ran 10 iterations |
+-----+
Press any key to continue . . .

```